

FIFO-RSHD – Remote shell prin fișiere FIFO

Autori:

Almășan Eduard-Ștefan

Feraru Răzvan-Mihai

Tiloiu Darius-Lucian

Grupa 144

Universitatea din București

Facultatea de Matematică și Informatică

Departamentul de Informatică

Instrumente și tehnici de bază în informatică (ITBI)

An universitar 2025–2026

Coordonator: Miruna Floroiu

Outline

- Descrierea problemei FIFO-RSHD și a contextului
- Specificația soluției: cum comunică clientul cu serverul
- Design arhitectural: client, server master, servere slave
- Implementare: scripturi shell, fișier de configurare, FIFO-uri
- Experimente: scenarii de test și metrici de evaluare
- Concluzii și pași următori pentru etapa de demo

Descrierea problemei

- Context: sisteme Unix/Linux și comunicare între procese (IPC)
- Dorim un „remote shell” simplificat, realizat exclusiv cu fișiere FIFO
- Există unul sau mai mulți clienți care vor să execute comenzi la distanță
- Există un server master (RSHD) și N servere slave care execută comenzile
- Toate cererile de la clienți pleacă către master printr-un FIFO cu nume cunoscut
- Răspunsul pentru fiecare client trebuie trimis înapoi doar către acel client
- Soluția modelează un sistem real tip client–server (similar conceptual cu SSH)

Specificația soluției

- Specificația soluției (variantă simplificată)
- Clientul trimite către server o comandă
- Cererea conține PID-ul clientului și comanda de executat
- Cererea este trimisă într-un FIFO principal (al serverului master)
- Serverul master primește cererea
- Masterul o trimite către un server slave, pe rând (Round-Robin)
- Serverul slave execută comanda
- Rezultatul comenzii este trimis înapoi clientului
- Fiecare client primește rezultatul în FIFO-ul lui propriu

Design

- Componenta Client:
 - citește comanda de la utilizator
 - află PID-ul propriu (\$\$) și creează FIFO-ul /tmp/server-reply-PID
 - formatează cererea și o scrie în well-known FIFO
 - citește rezultatul din FIFO-ul personal și îl afișează, apoi șterge FIFO-ul
- Serverul master RSHD:
 - citește fișierul de configurare (numele FIFO-ului master, N, FIFO-urile slave)
 - creează FIFO-urile și pornește N procese slave
 - pentru fiecare cerere: parsează PID + comandă și o trimite unui slave (Round-Robin)
- Serverele slave:
 - ascultă pe FIFO-ul propriu
 - execută comanda primită și scriu output-ul în FIFO-ul /tmp/server-reply-PID
- Model arhitectural: master–worker, cu master ca dispatcher și slave-uri ca workeri

Implementare

- Limbaj: scripturi bash rulate pe Linux
- Fișier de configurare (rshd.conf):
 - calea către well-known FIFO-ul masterului
 - numărul N de servere slave
 - lista FIFO-urilor pentru fiecare slave
- Scripturi planificate:
 - client.sh – interfața cu utilizatorul, trimiterea cererii, citirea răspunsului
 - master.sh – citirea cererilor și distribuirea lor către slave-uri
 - slave.sh – executarea comenzilor pentru clienți
- Mecanisme folosite: mkfifo, redirectionări (> și 2>&1), eval/sh pentru comenzi

Experimente

- Arhitectură de test:
 - un sistem Linux (fizic sau mașină virtuală)
 - 1 server master și N servere slave (ex. $N = 3$)
 - mai mulți clienți lansați din terminale diferite
- Scenarii:
 - comenzi simple: `ls`, `pwd`, `echo "hello"`
 - comenzi cu pipe-uri: `ls -l | wc -l`, `ps aux | grep bash`
 - comenzi invalide pentru a verifica mesajele de eroare
- Metrice orientative:
 - timp de răspuns de la trimiterea cererii până la ultimul output
 - comportamentul la mai mulți clienți simultan
 - corectitudinea output-ului față de rularea locală în shell

Concluzii

- Am analizat problema unui remote shell bazat pe fișiere FIFO (FIFO-RSHD)
- Am definit protocolul: formatul cererii și FIFO-urile personalizate pentru clienți
- Am propus o arhitectură master–multi-slave cu distribuire Round-Robin a cererilor
- Am stabilit componenta software și mecanismele care vor fi implementate în etapa a doua
- Am definit un plan de implementare și un set de experimente pentru testarea soluției
- Următorul pas: realizarea scripturilor bash și demonstrarea practică a sistemului FIFO-RSHD